

SE3332: Machine Learning Lab 1

赵楷越 522031910803

2025 年 3 月 29 日

1 Logistic Regression

1.1 模型与参数设置

下图中的模型实现了一个逻辑回归算法，使用交叉熵损失函数，用于二分类任务。它通过梯度下降法来优化模型的权重，以最小化损失函数。在训练过程中，模型会记录每次迭代的损失值和准确率，以便后续分析和可视化。数据加载时会检查数据是否为数值型，并为特征添加偏置项。模型训练完成后，会在测试集上进行预测并计算准确率。

```
# -----  
# 2. 定义逻辑回归类  
# -----  
class LogisticRegression:  
    def __init__(self, learning_rate=0.01, max_iter=1000, tol=1e-4, random_state=42):  
        self.learning_rate = learning_rate  
        self.max_iter = max_iter  
        self.tol = tol  
        self.random_state = random_state  
        self.weights = None  
        self.losses = [] # 初始化损失记录  
        self accuracies = [] # 初始化准确率记录
```

图 1: LR 训练类定义

超参数包括学习率、最大迭代次数、收敛阈值和随机种子。学习率决定了每次参数更新的步长，这里尝试了从 0.0001 到 0.01 的学习率，步长为 0.0001；最大迭代次数限制了训练的最大轮数，设为 1000 次；收敛阈值用于判断模型是否已经收敛，设为 1e-4；随机种子用于初始化权重时的随机数生成，设为 42，以保证结果的可重复性。评估指标主要采用准确率，即正确分类的样本数占总样本数的比例，用来衡量模型的分类性能。通过绘制损失曲线和准确率曲线，可以直观地观察模型在训练过程中的表现，以及是否达到预期的收敛效果。

```
# -----  
# 3. 训练模型并调参  
# -----  
# 从0.0001到0.01进行测试，步长0.0001  
learning_rates = np.arange(0.0001, 0.01, 0.0001)  
best_accuracy = 0  
best_learning_rate = None  
  
for lr in learning_rates:  
    print(f"尝试学习率: {lr:.4f}")  
    model = LogisticRegression(learning_rate=lr, max_iter=1000, tol=1e-4, random_state=42)  
    model.fit(X_train, Y_train)  
  
    # 在测试集上评估  
    Y_pred = model.predict(X_test)  
    accuracy = accuracy_score(Y_test, Y_pred)  
    print(f"测试集准确率: {accuracy:.4f}")  
  
    # 选择最佳学习率  
    if accuracy > best_accuracy:  
        best_accuracy = accuracy  
        best_learning_rate = lr  
  
print(f"最佳学习率: {best_learning_rate:.4f}, 测试集准确率: {best_accuracy:.4f}")
```

图 2: LR 多次训练

1.2 实验结果

总训练时间：0.3868s

最终，所有尝试的平均准确率：0.3144；实验下来最佳的学习率是 0.0008，测试集准确率为：0.8942，Bias: 0.4624。在这个实验中，损失函数曲线起初波动较大，随后逐渐下降并趋于稳定，表明模型的损失在训练过程中逐渐减小，最终达到一个较低的数值。准确率变化曲线起初也波动较大，但随着训练的进行，准确率逐渐提高并稳定在一个较高的水平，表明模型的预测能力在不断提升，最终达到了较好的准确率。总体来看，模型最终表现出良好的收敛性。

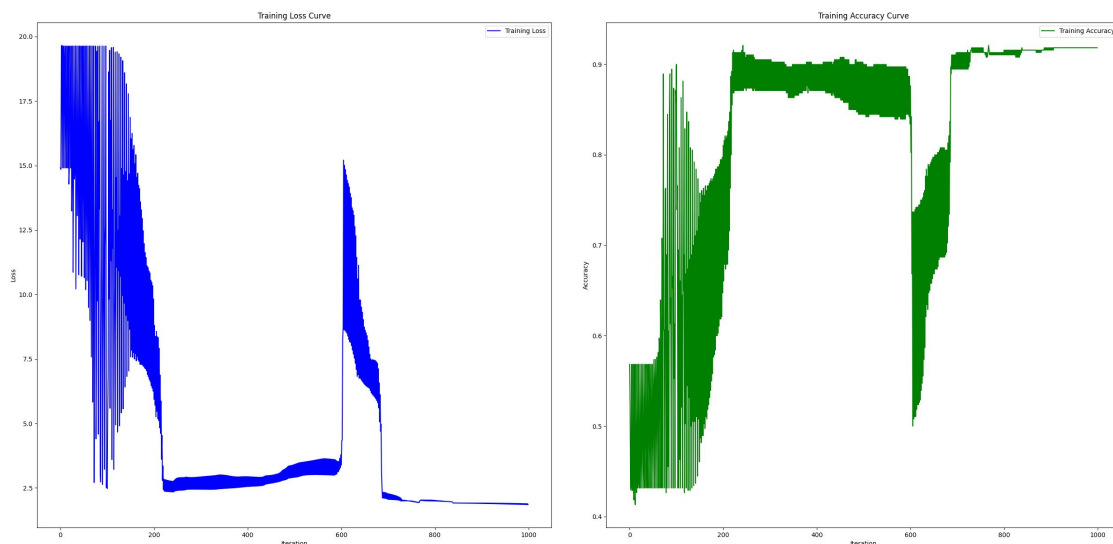


图 3: LR 损失函数及准确率曲线

以下图片展示了逻辑回归模型中各个特征的权重。正权重随着该值的增加、该特征模型预测结果更趋向于 1，负权重同理。如图所示，特征 2、6、7、9、12、17、20、24、25 和 27 的权重为正，对模型预测结果为 1 的影响较大；而特征 13、23 等的权重较大且为负，对模型预测为 0 的影响较大。

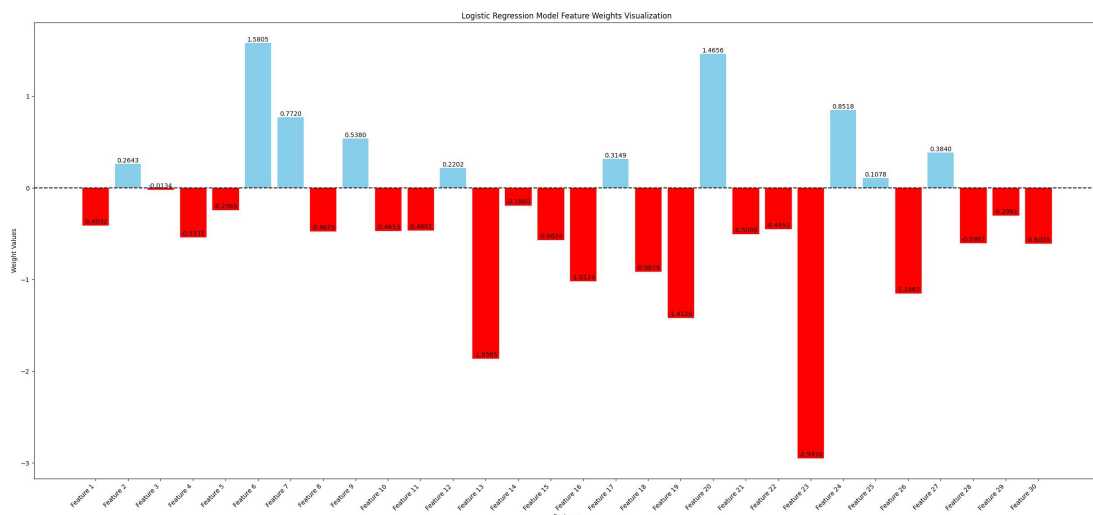


图 4: LR 权重结果

2 SVM

2.1 模型与参数设置

下图中的模型实现了一个基于梯度下降的支持向量机，用于二分类任务；模型的训练过程通过最小化带有 L2 正则化的 hinge loss 来更新权重；超参数包括学习率（learning_rate=0.001），它控制每次权重更新的步长；正则化参数（lambda_param=0.001），它控制模型的复杂度以防止过拟合；迭代次数（num_iterations=5000），它决定梯度下降的更新次数；评估指标包括训练集和测试集的损失值，用于衡量模型的拟合程度；以及训练集和测试集的准确率，用于衡量分类性能；模型在每次迭代中都会计算并记录训练集和测试集的损失和准确率，并在训练结束后绘制损失曲线和准确率曲线，以便直观地比较模型在训练集和测试集上的表现。

```
# -----  
# SVM Gradient Descent Implementation  
# -----  
class SVM:  
    def __init__(self, learning_rate=0.001, lambda_param=0.001, num_iterations=5000):  
        self.learning_rate = learning_rate # Learning rate  
        self.lambda_param = lambda_param # Regularization parameter  
        self.num_iterations = num_iterations # Number of iterations  
        self.weights = None # Model weights  
        self.train_loss_history = [] # To store training loss values  
        self.test_loss_history = [] # To store test loss values  
        self.train_accuracy_history = [] # To store training accuracy values  
        self.test_accuracy_history = [] # To store test accuracy values
```

图 5: SVM 训练类定义

```
# -----  
# Train SVM Model  
# -----  
svm = SVM(learning_rate=0.001, lambda_param=0.001, num_iterations=5000)
```

图 6: SVM 超参数

2.2 实验结果

总训练时间：2.41s

最终测试集准确率为：89.95%。这个模型的实验结果显示，训练损失和测试损失在初始阶段迅速下降，随后趋于平稳。训练损失和测试损失在后续大部分迭代过程中保持较低水平，表明模型在学习过程中表现良好。准确率曲线显示，训练准确率和测试准确率在初始阶段迅速上升，随后也趋于平稳，且两者接近，表明模型在训练和测试数据上的表现都较为一致。

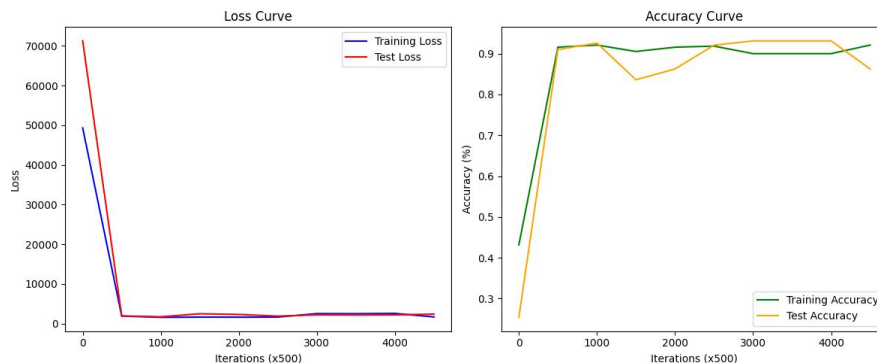


图 7: SVM 损失函数及准确率曲线

3 MLP

3.1 模型与参数设置

下图中的模型是一个简单的多层感知机（MLP）；模型的结构包括一个输入层、一个隐藏层和一个输出层；输入层的大小由训练数据的特征数决定，隐藏层有 10 个神经元，用 ReLU 激活函数来引入非线性；输出层有 1 个神经元，用 Sigmoid 激活函数将输出值映射到 0 到 1 之间，表示概率；模型的训练过程是通过前向传播计算输出值，然后用二分类交叉熵损失函数（BCELoss）计算损失；接着进行反向传播更新模型的参数，优化器使用 Adam 来调整参数；每训练一个 epoch，都会记录训练损失和测试损失；同时在测试集上评估模型，计算训练集和测试集的准确率，并记录下来；整个训练过程会重复指定的轮数（num_epochs），以优化模型的性能。模型的评估指标主要包括以下几个方面：首先是损失值，包括训练损失和测试损失，它们随着训练轮数的增加逐渐下降，表明模型在不断学习和优化；其次是准确率，包括训练准确率和测试准确率，它们反映了模型在训练集和测试集上的分类正确率，理想情况下两者都较高且接近，说明模型没有过拟合或欠拟合。

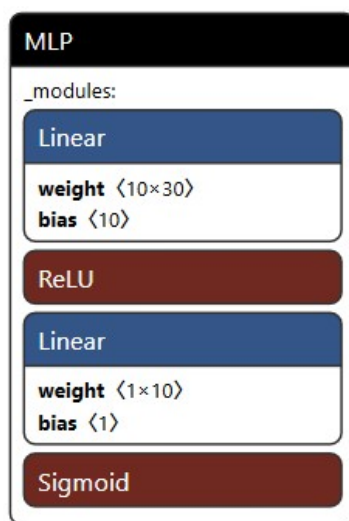


图 8: MLP 模型结构

这个模型中的超参数分别是：输入特征的数量（input_size），它由训练数据的列数决定，表示每个样本的特征数；隐藏层的神经元数量（hidden_size），这里设置为 10，代表模型中间隐藏层的神经元个数，用来提取输入特征的复杂模式；输出层的大小（output_size），设置为 1，模型输出一个概率值，根据概率值进行判断，表示这是一个二分类任务；学习率（learning_rate），设定为 0.01，控制模型每次参数更新的步长，影响训练的速度和稳定性；训练轮数（num_epochs），设置为 50，表示模型会对整个训练数据完整地训练 50 次。

```
# 超参数
input_size = X_train.shape[1]
hidden_size = 10
output_size = 1
learning_rate = 0.01
num_epochs = 50
```

图 9: MLP 超参数

3.2 实验结果

总训练时间：0.05s

这个模型的实验结果显示，随着训练的进行，损失逐渐下降，准确率逐渐上升。具体来说，损失从初始的 0.68 逐渐下降到 0.07，训练损失和测试损失的变化趋势基本一致，表明模型没有出现拟合或欠拟合的情况。同时，准确率从初始的低水平迅速上升并趋于稳定、最终达到接近百分百的准确率，实验结果显示模型在训练和测试数据上都有良好的表现。

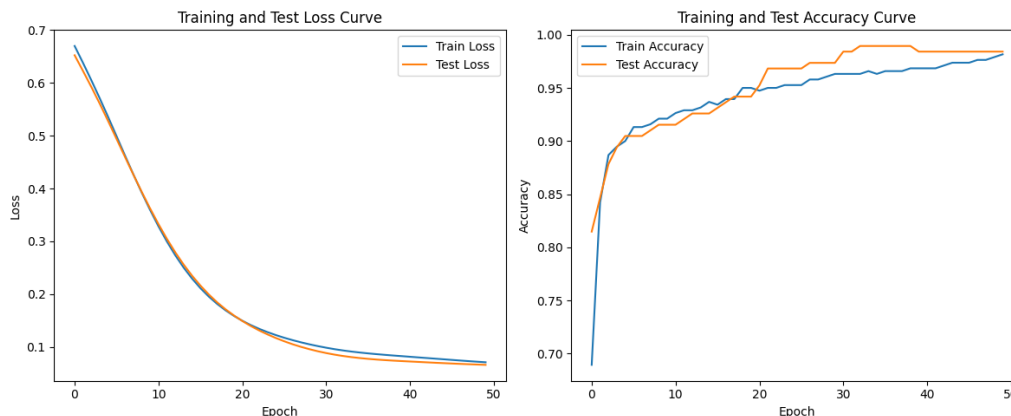


图 10: MLP 损失函数和准确率曲线

以下的混淆矩阵展示了模型的分类结果（其中 0 的结果为负、1 的结果为正）。在识别负样本方面，有 138 个正确识别为负样本，只有 3 个被错误地识别为正样本。在识别正样本方面，模型也表现不错，有 48 个正确识别为正样本，没有错误地识别为负样本。总体来看，这个模型在分类任务中表现相当不错。

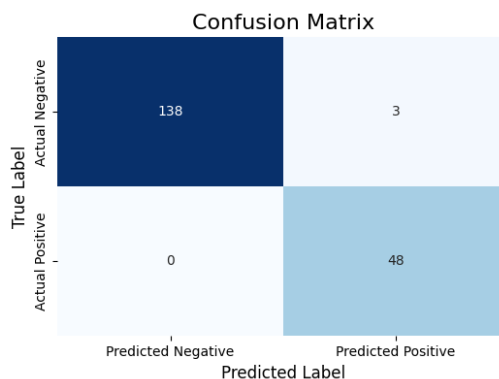


图 11: MLP 混淆矩阵

4 三种方法的对比与总结

现在对三种方法进行对比与总结：逻辑回归模型结构简单，训练速度快，但准确率相对较低，尤其是在复杂数据分布下表现有限，适合快速建模和小规模数据集；支持向量机通过最大化间隔来寻找最优分类超平面，结合 L2 正则化能有效防止过拟合，准确率中等、但训练时间相对较长；多层感知机（MLP）通过引入隐藏层和非线性激活函数，能够捕捉数据中的复杂模式，在准确率上表现最佳，几乎接近百分百，且训练时间最短，可能是由于使用了 GPU 进行训练，不过需要注意防止过拟合；总体来看，逻辑回归适合快速建模和简单任务，支持向量机在中小规模数据集上表现均衡，而多层感知机在准确率上具有明显优势，但需要更多的数据和调参工作以避免过拟合；在实际应用中，可以根据数据规模、复杂度和计算资源选择合适的方法。

5 参考

1. <https://pytorch.org/tutorials/>